BACKEND TEAM PROPOSAL

Backend Architecture & Implementation Plan

SUT Campus & Life Companion

1. Backend Architecture Overview

The backend is organized into six API modules, with clear separation between request handling, data validation, and application logic. The goal is to let both backend engineers work in parallel with well-defined ownership and minimal code conflicts.

The backend team is responsible for implementing the API endpoints, request/response schemas, validation, and internal service functions. Integration with external services is handled through a clearly defined interface when the required service contract is available.

Backend API Modules

AuthAPI	RideAPI	ChatAPI	ReportAPI	RouteAPI	ChatbotAPI
register login getProfile	createRide searchRides requestJoin respond	openChat sendMessage	submitReport getWarnings	findRoute getStops	ask

Technology stack: Python · FastAPI · Pydantic · Uvicorn

The stack supports API development, request validation, and running the backend application. The scope of this proposal is limited to the backend team's own implementation work.

2. Task Distribution and Ownership

The six API modules are split evenly between the two backend engineers. Each engineer owns three APIs, including their routers, schemas, validation, and endpoint behavior.

OWNER	API MODULES	RESPONSIBILITIES
Abdullah's teammate (AI-Connected APIs)	RouteAPI ChatbotAPI ReportAPI	Route search and stop information; transport-related chatbot requests; reports and warning retrieval.
Abdullah Mohamed (Core Backend)	AuthAPI RideAPI ChatAPI	Registration, login and profiles; ride creation and search; ride-join requests and direct messaging.

API Responsibilities

API	METHODS	BACKEND SCOPE
RouteAPI	GET / POST	Accepts a starting point and destination; validates the request; calls the route-integration function; returns route and stop information according to the agreed API contract.
ChatbotAPI	POST	Accepts and validates a user's transport-related question, passes it to the chatbot-integration function, and returns the response.
ReportAPI	POST / GET	Accepts user reports and exposes warning information through defined endpoints. Any moderation decision is handled through an agreed service interface.

Scope clarification: This document covers only the work of the two-person Backend Team. It does not define or assign tasks to other teams.

3. Execution Plan

Phase 1 — API Development

Create FastAPI routers and Pydantic schemas for the assigned endpoints. Define request and response models, validate incoming data, return consistent status codes, and handle expected errors.

Phase 2 — Backend Service Layer

Keep business logic in service modules rather than placing all logic inside route handlers. For the AI-connected endpoints, create a dedicated module such as `services/ai_service.py` to centralize calls to the relevant service interfaces. The exact request fields and response models should follow the interface contract agreed for each integration.

Backend Request Flow

FRONTEND	FASTAPI ROUTER	BACKEND SERVICE	RESPONSE
Sends request	Validates input	Runs business logic / calls defined interface	Returns structured data

Frontend → FastAPI Router → Validation → Backend Service / Integration Interface → Structured Response

Internal Service Functions

FUNCTION	API	BACKEND RESPONSIBILITY
suggestRoute	RouteAPI	Pass origin and destination through the agreed interface and format the suggested-route result.
estimateCostTime	RouteAPI	Pass route details through the agreed interface and format estimated cost and travel time.
answerChat	ChatbotAPI	Pass the user's question through the agreed interface and format the answer.

Deliverable
A clearly organized backend implementation plan for Abdullah and his teammate, with defined API ownership, endpoint responsibilities, validation, and service-layer structure.